RESONATE — Build Plan

Scripture for the stories people are already telling • Kaggle: *Scripture in New Frontiers* ($10,000) • A personalized, verified, safety-gated Scripture engine with a temporal memory graph

Prize	Keys open	Due	Frontier
$10,000 single winner	**6 Jul 2026**	**1 Aug 2026** 10:29 IST	**Creator tools**

The idea in one line

Creators record honest content every day — burnout, doubt, joy, grief — and Scripture never meets it. Resonate reads a creator's own transcript, finds the verse their story was already echoing, fetches the **real verse text live from YouVersion**, and weaves it back into their content as a caption, spoken line, or auto-cut short. The **engine** is invisible plumbing; the **experience** lives natively in one frontier: **creator tools**.

Why this is built to win (mapped to the 100-pt rubric)

Impact & Vision 40 pts	Verified, safe Scripture inside a creator's own content — **multilingual** (YouVersion's ~1,500 translations) so it can reach billions who don't read English. Every edited video becomes a distribution channel. That's scale without needing billions of users on day one.
Video & Story 30 pts	A built-in 3-act arc: a creator's raw vulnerable moment → the verse surfacing inside their own footage → a shareable short landing on someone who needed it. The **series-memory** reveal ("you've returned to this 4 times") is the moment that sticks.
Technical Depth 30 pts	Hybrid retrieval (dense + BM25 + tags) fused with **RRF**, a **temporal memory graph**, an **LLM verify** step constrained to a vetted verse set, anti-hallucination via YouVersion, confidence/abstention, safety routing, and an **evaluation harness with real metrics**. Impossible to dismiss as faked.

What a valid submission needs (don't miss one)

1. Writeup	≤500-word project report (title, subtitle, analysis)
2. Cover image	required in the media gallery
3. Public notebook	code as a public Kaggle notebook (no login/paywall)
4. Video	≤3 min, public on YouTube — the primary judging lens
5. Project link	live public demo, or public GitHub repo + setup steps

Architecture & engine logic

Gloo AI = understanding + words (segment emotion, verify/select the best candidate with a rationale, write the bridge, run safety). **YouVersion** = ground-truth verse text, fetched live in the creator's language — never recited by the LLM. **The engine** = the matchmaker + memory in between (our IP).

raw text → **segment** (Gloo) → beats → **encode** → **hybrid retrieve** (dense + BM25 + tags, fused with **RRF**) → **memory re-rank** (context graph) → **LLM verify** (Gloo, constrained) → **safety gate** → **confidence/abstain** → **fetch text** (YouVersion) → **bridge** (Gloo) → deliver → **remember**

Two ideas borrowed from GitNexus — built ourselves, no license baggage

- **Reciprocal Rank Fusion (RRF)** to merge the three retrievers — ~15 lines, no graph DB. `RRF(v) = Σ 1/(k + rank_r(v))`, k≈60.
- **A per-user context graph** (beats ↔ themes ↔ verses over time) for recency, theme-fatigue, narrative arcs, and the series-memory feature.

(GitNexus itself is PolyForm **Noncommercial** — unsafe to depend on for a cash-prize, commercial-scale entry. We take the ideas, not the code.)

Storage & caching layer (Redis-backed, local fallback)

One memory interface, two backends: **local** (in-memory + JSON/SQLite) runs offline and in the notebook; **Redis** is the production backend doing three jobs in one fast store — a **vector index** (HNSW) for dense retrieval, the **episodic context graph**, and a **semantic cache** for Gloo calls (lower latency + cost). Redis never blocks the demo: if it's down, the engine falls back to local automatically. (This is Redis's *"Context is all you need"* thesis made concrete — and a clean scale story for Impact.)

The fit score (after fusion)

final = **w_rrf**·norm(RRF) + **w_tone**·tone_fit(intensity) – **w_recent**·recently_used – **w_repeat**·theme_fatigue + **w_arc**·narrative_continuity

tone_fit matches posture to intensity (deep grief → a *comfort* verse, not "rejoice!"). The three memory terms make it *living*: no repeats, no over-hammering one theme, and continuity with the person's recurring journey.

Proof it works (not just a demo)

- **Evaluation harness**: a labeled set (context → themes/verses) scored on Theme-F1, Recall@k, Precision@1, no-repeat rate, and safety recall.
- **LLM-as-judge** tone-appropriateness score (1–5) via Gloo.
- **Safety first**: crisis/self-harm beats route to a human-help card — never auto-answered with a verse. Built before anything flashy.
- **Confidence & abstention**: when unsure, soften or stay silent rather than force a tone-deaf hit.

The build plan — 8 phases (each ends in a GitHub push)

Statuses: **Done** · **Now** · To do. **Minimum winning path:** Phases 0–2, 4, 5, 6, 7. Phase 3 (memory graph) & multilingual are high-value differentiators; the creator-export stretch is cut first if time runs short.

Phase 0 — Offline engine core

Now (no keys needed)

Goal: prove the full context→verse pipeline runs end-to-end, fully offline.

Step	Status
Repo scaffold + README + ENGINE-DESIGN.md	**Done**
Curated verse shortlist (data/verses.json, 36 verses, tags only — no verse text)	**Done**
Embedding interface + verse store (TF-IDF v0, swappable for dense)	**Done**
Config + mock/live adapter switch	**Done**
Memory store interface + per-user context graph, local backend (runs offline)	**Now**
Mock Gloo (segment, verify, bridge, safety) + Mock YouVersion (public-domain samples)	**Now**
Engine orchestrator: hybrid retrieve + RRF + tone + memory re-rank + abstain	**Now**
CLI demo + verify: beats → verse → bridge, memory across episodes, safety hold	To do

■ **Milestone:** push #1 — "Offline engine core working"

Phase 1 — Real APIs + multilingual

Week 1 (after 6 Jul)

Goal: same pipeline, now powered by the two required live APIs, in multiple languages.

Step	Status
Attend webinar; note key signup + per-translation licensing rules	To do
Obtain Gloo API key + YouVersion app key	To do
LiveYouVersion: verified fetch by reference; confirm response shape + headers	To do
LiveGloo: structured segmentation (JSON) + verify/select + bridge	To do
Translation / language selection (multilingual delivery)	To do
Flip RESONATE_MODE=live; re-run demo against real APIs	To do

■ **Milestone:** push #2 — "Live Gloo + YouVersion + multilingual"

Phase 2 — State-of-the-art retrieval & reasoning

Week 1–2

Goal: make the matching genuinely best-in-class and defensible.

Step	Status
Dense embeddings (sentence-transformers) replace TF-IDF	To do
BM25 sparse retriever	To do

RRF fusion of dense + sparse + tag retrievers; tune k and weights	To do
Redis backend behind adapter: vector index (HNSW) + semantic cache for Gloo calls	To do
LLM verify/select (Gloo picks best from shortlist + rationale, constrained)	To do
Confidence scoring + abstention threshold	To do
Expand verse shortlist to ~200–300, reviewed for accuracy	To do

■ **Milestone:** push #3 — "Hybrid RRF retrieval + LLM verify"

Phase 3 — Context graph & series memory

Week 2

Goal: the temporal memory that makes Resonate feel alive (the differentiator).

Step	Status
Full per-user temporal graph (User/Episode/Beat/Theme/Verse), persisted	To do
Persist the context graph in Redis (episodic memory); automatic local fallback	To do
Recurring-theme arc detection across episodes	To do
Series-memory insights ("returned to X 4 times") + compilation data	To do

■ **Milestone:** push #4 — "Temporal context graph + series memory"

Phase 4 — Evaluation & safety

Week 2

Goal: prove quality with numbers and protect real people.

Step	Status
Labeled eval set (context → themes / gold verses)	To do
Metrics: Theme-F1, Recall@k, Precision@1, no-repeat rate	To do
LLM-as-judge tone-appropriateness score (1–5)	To do
Crisis test set + safety recall; hardened routing + real resources card	To do

■ **Milestone:** push #5 — "Eval harness + hardened safety"

Phase 5 — Web UI / live demo

Week 2–3

Goal: the public Project Link judges can click — and the screen you'll film.

Step	Status
FastAPI app serving a polished single-page UI	To do
Show beats, fused candidates, fit breakdown, context-graph view, series memory	To do
Translation picker + text-to-speech playback	To do
Deploy publicly (Render / Railway / HF Spaces)	To do

■ **Milestone:** push #6 — "Live demo deployed"

Phase 6 — Kaggle notebook

Week 3

Goal: the required public notebook that verifies the tech is real.

Step	Status
Public notebook: engine walkthrough + real API calls with visible responses	To do
Include the evaluation metrics table	To do
Clear setup instructions in README	To do

■ **Milestone:** push #7 — "Public Kaggle notebook"

Phase 7 — Story: video + writeup + submit

Week 4

Goal: win the 70% that is vision + storytelling, then submit before the deadline.

Step	Status
3-min script: problem → engine → verse in context → share → series memory → multilingual	To do
Shoot + edit; upload public to YouTube	To do
≤500-word technical writeup	To do
Cover image + media gallery	To do
Final submission on Kaggle before 1 Aug, 10:29 IST	To do

■ **Milestone:** push #8 — "Submission-ready" + submit

Stretch — Creator export (cut first if time runs short)

Render branded caption cards onto a clip · auto-cut a 15–30s vertical short around the strongest beat · series-memory compilation reel from the creator's own footage.

GitHub workflow

- **You create** a **public** repo (e.g. `resonate`); I wire the remote and push. Public repo is non-negotiable for judging. (gh CLI isn't installed locally.)
- Short feature branches → merge to `main` each phase. Tag milestones `v0.1 … v0.8`.
- `.gitignore`: `.env`, `__pycache__/`, `.venv/`, `node_modules/`, `out/`.
- **Never commit keys.** Commit `.env.example` only.
- MIT `LICENSE` + README with copy-paste setup. Commit messages name the phase/step, so the history reads as this plan.

Decisions to confirm early (cheap now, expensive later)

- **Translation licensing** — verify at the webinar that your demo translations carry no non-commercial block.
- **Gloo auth / base URL / model id** — confirm with a real key the moment you have one.
- **YouVersion passage endpoint** — confirm exact reference syntax, response shape, rate limits.
- **Redis hosting** — a free managed tier (Redis Cloud / Upstash) for the deployed demo; local fallback means this is never on the critical path.
- **Scope discipline** — protect demo + video time above all. Cut the creator-export stretch before you cut polish on the demo or the story.

Already in the repo (as of this plan)

`README.md`, `ENGINE-DESIGN.md` (v2), `data/verses.json`, and the engine package (`config / models / embeddings / verses`). Remaining Phase-0 files — the context-graph `memory.py`, providers, `engine.py`, and the demo — are the next build step.